Testen mit JUnit (JUnit-Basics)

TL;DR
In JUnit 4 und 5 werden Testmethoden mit Hilfe der Annotation `@Test` ausgezeichnet. Über die verschiedenen `assert*()`-Methoden kann das Testergebnis mit dem erwarteten Ergebnis verglichen werden und entsprechend ist der Test "grün" oder "rot". Mit den verschiedenen `assume*()`-Methoden kann dagegen geprüft werden, ob eventuelle Vorbedingungen für das Ausführen eines Testfalls erfüllt sind - anderenfalls wird der Testfall dann übersprungen.
Mit Hilfe von `@Before` und `@After` können Methoden gekennzeichnet werden, die jeweils vor jeder Testmethode und nach jeder Testmethode aufgerufen werden. Damit kann man seine Testumgebung auf- und auch wieder abbauen (JUnit 4).
Erwartete Exceptions lassen sich in JUnit 4 mit einem Parameter `expected` in der Annotation `@Test` automatisch prüfen: `@Test(expected=package.Exception.class)`. In JUnit 4 besteht die Möglichkeit, Testklassen zu Testsuiten zusammenzufassen und gemeinsam laufen zu lassen.

Videos

- VL JUnit Basics
- Demo assume() vs. assert()
- Demo Parametrisierte Tests mit JUnit4
- Demo Parametrisierte Tests mit JUnit5

Unterlagen

- PDF-Version

JUnit: Ergebnis prüfen

Klasse `org.junit.Assert` enthält diverse **statische** Methoden zum Prüfen:

```
// Argument muss true bzw. false sein
void assertTrue(boolean);
void assertFalse(boolean);

// Gleichheit im Sinne von equals()
void assertEquals(Object, Object);

// Test sofort fehlschlagen lassen
void fail();

...
```

To "assert" or to "assume"?

- Mit `assert*` werden Testergebnisse geprüft

  - Test wird ausgeführt
  - Ergebnis: OK, Failure, Error

- Mit `assume*` werden Annahmen über den Zustand geprüft
  - Test wird abgebrochen, wenn Annahme nicht erfüllt
  - Prüfen von Vorbedingungen: Ist der Test hier ausführbar/anwendbar?

Beispiel: junit4.TestAssume (https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture/blob/master/lecture/quality/src/junit4/TestAssume.java)

Setup und Teardown: Testübergreifende Konfiguration

```
private Studi x;

@Before
public void setUp() { x = new Studi(); }

@Test
public void testToString() {
    // Studi x = new Studi();
    assertEquals(x.toString(), "Heinz (15cps)");
}
```

`@Before` wird **vor jeder** Testmethode aufgerufen
`@BeforeClass` wird **einmalig** vor allen Tests aufgerufen (**`static`**!)
`@After` wird **nach jeder** Testmethode aufgerufen
`@AfterClass` wird **einmalig** nach allen Tests aufgerufen (**`static`**!)

In JUnit 5 wurden die Namen dieser Annotationen leicht geändert:

JUnit 4	JUnit 5
`@Before`	`@BeforeEach`
`@After`	`@AfterEach`
`@BeforeClass`	`@BeforeAll`
`@AfterClass`	`@AfterAll`

Beispiel für den Einsatz von `@Before`

Annahme: **alle/viele** Testmethoden brauchen **neues** Objekt `x` vom Typ `Studi`

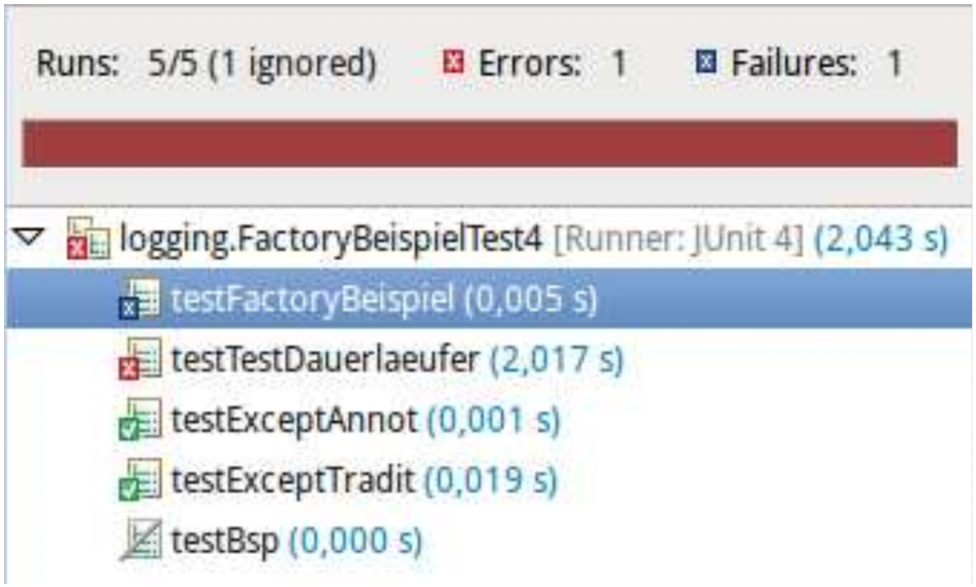
```
private Studi x;

@Before
```

```
4   public void setUp() {
5       x = new Studi("Heinz", 15);
6   }
7
8   @Test
9   public void testToString() {
10      // Studi x = new Studi("Heinz", 15);
11      assertEquals(x.toString(), "Name: Heinz, credits: 15");
12  }
13
14  @Test
15  public void testGetName() {
16      // Studi x = new Studi("Heinz", 15);
17      assertEquals(x.getName(), "Heinz");
18  }
```

Ignorieren von Tests

- Hinzufügen der Annotation `@Ignore`
- Alternativ mit Kommentar: `@Ignore("Erst im nächsten Release")`

```
1  @Ignore("Warum ignoriert")
2  @Test
3  public void testBsp() {
4      Bsp x = new Bsp();
5      assertTrue(x.isTrue());
6  }
```

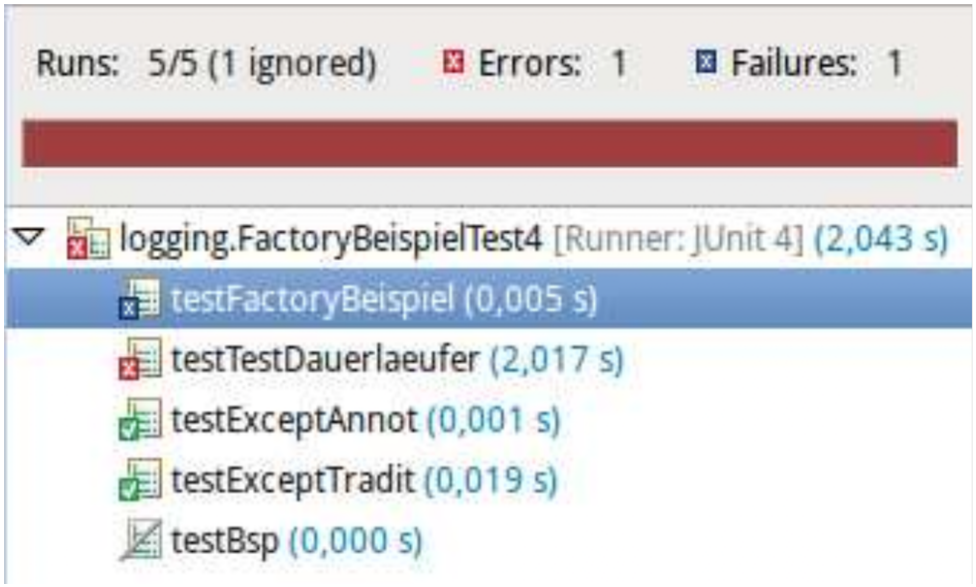


In JUnit 5 wird statt der Annotation `@Ignore` die Annotation `@Disabled` mit der selben Bedeutung verwendet. Auch hier lässt sich als Parameter ein String mit dem Grund für das Ignorieren des Tests hinterlegen.

Vermeidung von Endlosschleifen: Timeout

- Testfälle werden nacheinander ausgeführt
- Test mit Endlosschleife würde restliche Tests blockieren
- Erweitern der `@Test`-Annotation mit Parameter "`timeout`":
  => `@Test(timeout=2000)` (Zeitangabe in Millisekunden)

```
@Test(timeout = 2000)
void testTestDauerlaeufer() {
    while (true) { ; }
}
```



In JUnit 5 hat die Annotation `@Test` keinen `timeout`-Parameter mehr. Als Alternative bietet sich der Einsatz von `org.junit.jupiter.api.Assertions.assertTimeout` an. Dabei benötigt man allerdings *Lambda-Ausdrücke* (Verweis auf spätere VL):

```
@Test
void testTestDauerlaeufer() {
    assertTimeout(ofMillis(2000), () -> {
        while (true) { ; }
    });
}
```

(Beispiel von oben mit Hilfe von JUnit 5 formuliert)

Test von Exceptions: Expected

Traditionelles Testen von Exceptions mit **`try`** und **`catch`**:

```
@Test
```

```
2  public void testExceptTradit() {
3      try {
4          int i = 0 / 0;
5          fail("keine ArithmeticException ausgeloest");
6      } catch (ArithmeticException aex) {
7          assertNotNull(aex.getMessage());
8      } catch (Exception e) {
9          fail("falsche Exception geworfen");
10     }
11 }
```

Der `expected`-Parameter für die `@Test`-Annotation in JUnit 4 macht dies deutlich einfacher: `@Test(expected = MyException.class)` => Test scheitert, wenn diese Exception **nicht** geworfen wird

```
1  @Test(expected = java.lang.ArithmeticException.class)
2  public void testExceptAnnot() {
3      int i = 0 / 0;
4  }
```

In JUnit 5 hat die Annotation `@Test` keinen `expected`-Parameter mehr. Als Alternative bietet sich der Einsatz von `org.junit.jupiter.api.Assertions.assertThrows` an. Dabei benötigt man allerdings *Lambda-Ausdrücke* (Verweis auf spätere VL):

```
1  @Test
2  public void testExceptAnnot() {
3      assertThrows(java.lang.ArithmeticException.class, () -> {
4          int i = 0 / 0;
5      });
6  }
```

(Beispiel von oben mit Hilfe von JUnit 5 formuliert)

Parametrisierte Tests

Manchmal möchte man den selben Testfall mehrfach mit anderen Werten (Parametern) durchführen.

```
1  class Sum {
2      public static int sum(int i, int j) {
3          return i + j;
4      }
5  }
6
7  class SumTest {
8      @Test
9      public void testSum() {
10         Sum s = new Sum();
11         assertEquals(s.sum(1, 1), 2);
12     }
13     // und mit (2,2, 4), (2,2, 5), ...????
14 }
```

Prinzipiell könnte man dafür entweder in einem Testfall eine Schleife schreiben, die über die verschiedenen Parameter iteriert. In der Schleife würde dann jeweils der Aufruf der zu testenden Methode und das gewünschte

Assert passieren. Alternativ könnte man den Testfall entsprechend oft duplizieren mit jeweils den gewünschten Werten.

Beide Vorgehensweisen haben Probleme: Im ersten Fall würde die Schleife bei einem Fehler oder unerwarteten Ergebnis abbrechen, ohne dass die restlichen Tests (Werte) noch durchgeführt würden. Im zweiten Fall bekommt man eine unnötig große Anzahl an Testmethoden, die bis auf die jeweiligen Werte identisch sind (Code-Duplizierung).

Parametrisierte Tests mit JUnit 4

JUnit 4 bietet für dieses Problem sogenannte "parametrisierte Tests" an. Dafür muss eine Testklasse in JUnit 4 folgende Bedingungen erfüllen:

1. Die Testklasse wird mit der Annotation `@RunWith(Parameterized.class)` ausgezeichnet.
2. Es muss eine öffentliche statische Methode geben mit der Annotation `@Parameters`. Diese Methode liefert eine Collection zurück, wobei jedes Element dieser Collection ein Array mit den Parametern für einen Durchlauf der Testmethoden ist.
3. Die Parameter müssen gesetzt werden. Dafür gibt es zwei Varianten:
   - (A) Für jeden Parameter gibt es ein öffentliches Attribut. Diese Attribute müssen mit der Annotation `@Parameter` markiert sein und können in den Testmethoden normal genutzt werden. JUnit sorgt dafür, dass für jeden Eintrag in der Collection aus der statischen `@Parameters`-Methode diese Felder gesetzt werden und die Testmethoden aufgerufen werden.
   - (B) Alternativ gibt es einen Konstruktor, der diese Werte setzt. Die Anzahl der Parameter im Konstruktor muss dabei exakt der Anzahl (und Reihenfolge) der Werte in jedem Array in der von der statischen `@Parameters`-Methode gelieferten Collection entsprechen. Der Konstruktor wird für jeden Parametersatz einmal aufgerufen und die Testmethoden einmal durchgeführt.

Letztlich wird damit das Kreuzprodukt aus Testmethoden und Testdaten durchgeführt.

(A) Parametrisierte Tests: Konstruktor (JUnit 4)

```java
@RunWith(Parameterized.class)
public class SumTestConstructor {
    private final int s1;
    private final int s2;
    private final int erg;

    public SumTestConstructor(int p1, int p2, int p3) { s1 = p1;  s2 = p2;  erg = p3; }

    @Parameters
    public static Collection<Object[]> values() {
        return Arrays.asList(new Object[][] { { 1, 1, 2 }, { 2, 2, 4 }, { 2, 2, 5 } });
    }

    @Test
    public void testSum() {
        assertEquals(Sum.sum(s1, s2), erg);
    }
```

```
}
```

(B) Parametrisierte Tests: Parameter (JUnit 4)

```
@RunWith(Parameterized.class)
public class SumTestParameters {

    @Parameter(0)  public int s1;
    @Parameter(1)  public int s2;
    @Parameter(2)  public int erg;

    @Parameters
    public static Collection<Object[]> values() {
        return Arrays.asList(new Object[][] { { 1, 1, 2 }, { 2, 2, 4 }, { 2, 2,
            5 } });
    }

    @Test
    public void testSum() {
        assertEquals(Sum.sum(s1, s2), erg);
    }
}
```

Beispiel: junit4.SumTestConstructor, junit4.SumTestParameters (https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture/tree/master/lecture/quality/src/junit4/)

Parametrisierte Tests mit JUnit 5

In JUnit 5 werden parametrisierte Tests mit der Annotation `@ParameterizedTest` gekennzeichnet (statt mit `@Test`).

Mit Hilfe von `@ValueSource` kann man ein einfaches Array von Werten (Strings oder primitive Datentypen) angeben, mit denen der Test ausgeführt wird. Dazu bekommt die Testmethode einen entsprechenden passenden Parameter:

```
@ParameterizedTest
@ValueSource(strings = {"wuppie", "fluppie", "foo"})
void testWuppie(String candidate) {
    assertTrue(candidate.equals("wuppie"));
}
```

Alternativ lassen sich als Parameterquelle u.a. Aufzählungen (`@EnumSource`) oder Methoden (`@MethodSource`) oder auch Komma-separierte Daten (`@CsvSource`) angeben.

Das obige Beispiel aus JUnit 4.x könnte mit Hilfe von `@CsvSource` so in JUnit 5.x umgesetzt werden:

```
public class SumTest {
    @ParameterizedTest
    @CsvSource(textBlock = """
            # s1,  s2,  s1+s2
            0,     0,   0
            10,    0,   10
            0,     11,  11
            -2,    10,  8
```

```
9                 """)
10      public void testSum(int s1, int s2, int erg) {
11          assertEquals(Sum.sum(s1, s2), erg);
12      }
13  }
```

Beispiel: junit5.TestValueSource, junit5.TestMethodSource (https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture/tree/master/lecture/quality/src/junit5/)

Testsuiten: Tests gemeinsam ausführen (JUnit 4)

Eclipse: `New` > `Other` > `Java` > `JUnit` > `JUnit Test Suite`

```
1  import org.junit.runner.RunWith;
2  import org.junit.runners.Suite;
3  import org.junit.runners.Suite.SuiteClasses;
4
5  @RunWith(Suite.class)
6  @SuiteClasses({
7      // Hier kommen alle Testklassen rein
8      PersonTest.class,
9      StudiTest.class
10 })
11
12 public class MyTestSuite {
13     // bleibt leer!!!
14 }
```

Testsuiten mit JUnit 5

In JUnit 5 gibt es zwei Möglichkeiten, Testsuiten zu erstellen:

- `@SelectPackages`: Angabe der Packages, die für die Testsuite zusammengefasst werden sollen
- `@SelectClasses`: Angabe der Klassen, die für die Testsuite zusammengefasst werden sollen

```
1  @RunWith(JUnitPlatform.class)
2  @SelectClasses({StudiTest5.class, WuppieTest5.class})
3  public class MyTestSuite5 {
4      // bleibt leer!!!
5  }
```

Zusätzlich kann man beispielsweise mit `@IncludeTags` oder `@ExcludeTags` Testmethoden mit bestimmten Tags einbinden oder ausschließen. Beispiel: Schließe alle Tests mit Tag “develop” aus: `@ExcludeTags("develop")`. Dabei wird an den Testmethoden zusätzlich das Tag `@Tag` verwendet, etwas `@Tag("develop")`.

Achtung: Laut der offiziellen Dokumentation (Abschnitt “4.4.4. Test Suite”) gilt zumindest bei der Selection über `@SelectPackages` der Zwang zu einer Namenskonvention: Es werden dabei nur Klassen gefunden, deren Name mit `Test` beginnt oder endet! Weiterhin werden Testsuites mit der Annotation

`@RunWith(JUnitPlatform.class)` **nicht** auf der "JUnit 5"-Plattform ausgeführt, sondern mit der JUnit 4-Infrastuktur!

Best Practices

1. Ein Testfall behandelt exakt eine Idee/ein Szenario. Das bedeutet auch, dass man in der Regel nur ein bis wenige `assert*` pro Testmethode benutzt.

   (Wenn man verschiedene Ideen in eine Testmethode kombiniert, wird der Testfall unübersichtlicher und auch auch schwerer zu warten.

   Außerdem können so leichter versteckte Fehler auftreten: Das erste oder zweite oder dritte `assert*` schlägt fehl - und alle dahinter kommenden `assert*` werden nicht mehr ausgewertet!)

2. Wenn die selbe Testidee mehrfach wiederholt wird, sollte man diese Tests zu einem parametrisierten Test zusammenfassen.

   (Das erhöht die Lesbarkeit drastisch - und man läuft auch nicht in das Problem der Benennung der Testmethoden.)

3. Es wird nur das Verhalten der öffentlichen Schnittstelle getestet, nicht die inneren Strukturen einer Klasse oder Methode.

   (Es ist verlockend, auch private Methoden zu testen und in den Tests auch die Datenstrukturen o.ä. im Blick zu behalten und zu testen. Das führt aber zu sehr "zerbrechlichen" (*brittle*) Tests: Sobald sich etwas an der inneren Struktur ändert, ohne dass sich das von außen beobachtbare Verhalten ändert und also die Klasse/Methode immer noch ordnungsgemäß funktioniert, gehen all diese "internen" Tests kaputt. Nicht ohne Grund wird in der objektorientierten Programmierung mit Kapselung (Klassen, Methoden, ...) gearbeitet.)

4. Von Setup- und Teardown-Methoden sollte eher sparsam Gebrauch gemacht werden.

   (Normalerweise folgen wir in der objektorientierten Programmierung dem DRY-Prinzip (Don't repeat yourself). Entsprechend liegt es nahe, häufig benötigte Elemente in einer Setup-Methode zentral zu initialisieren und ggf. in einer Teardown-Methode wieder freizugeben.

   Das führt aber speziell bei Unit-Tests dazu, dass die einzelnen Testmethoden schwerer lesbar werden: Sie hängen von einer gemeinsamen, zentralen Konfiguration ab, die man üblicherweise nicht gleichzeitig mit dem Code der Testmethode sehen kann (begrenzter Platz auf der Bildschirmseite).

   Wenn nun in einem oder vielleicht mehreren Testfällen der Wunsch nach einer leicht anderen Konfiguration auftaucht, muss man die gemeinsame Konfiguration entsprechend anpassen bzw. erweitern. Dabei muss man dann aber *alle* anderen Testmethoden mit bedenken, die ja ebenfalls von dieser Konfiguration abhängen! Das führt in der Praxis dann häufig dazu, dass die gemeinsame Konfiguration sehr schnell sehr groß und verschachtelt und entsprechend unübersichtlich wird.

   Jede Änderung an dieser Konfiguration kann leicht einen oder mehrere Testfälle kaputt machen (man hat ja i.d.R. nie alle Testfälle gleichzeitig im Blick), weshalb man hier unbedingt mit passenden `assume*` arbeiten muss - aber dann kann man eigentlich auch stattdessen die Konfiguration direkt passend für den jeweiligen Testfall in der jeweiligen Testmethode erledigen!)

5. Wie immer sollten auch die Namen der Testmethoden klar über ihren Zweck Auskunft geben.

   (Der Präfix “test” wird seit JUnit 4.x nicht mehr benötigt, aber dennoch ist es in vielen Projekten Praxis, diesen Präfix beizubehalten - damit kann man in der Package-Ansicht in der IDE leichter zwischen den “normalen” und den Testmethoden unterscheiden.)

Diese Erfahrungen werden ausführlich in (Winters, Manshreck, und Wright 2020, pp. 231-256) diskutiert.

Wrap-Up

JUnit als Framework für (Unit-) Tests; hier JUnit 4 (mit Ausblick auf JUnit 5)

- Testmethoden mit Annotation `@Test`
- `assert` (Testergebnis) vs. `assume` (Testvorbedingung)
- Aufbau der Testumgebung `@Before`
- Abbau der Testumgebung `@After`
- Steuern von Tests mit `@Ignore` oder `@Test(timout=XXX)`
- Exceptions einfordern mit `@Test(expected=package.Exception.class)`
- Tests zusammenfassen zu Testsuiten

Zum Nachlesen

- vogella GmbH (2021)
- The JUnit Team (2022)
- Kleuker (2019)
- Osherove (2014)
- Spillner und Linz (2012)
- Thies, Noelke, und Ungerc (o. J.)

Lernziele

- k3: Steuern von Tests (ignorieren, zeitliche Begrenzung)
- k3: Prüfung von Exceptions
- k3: Aufbau von Testsuiten mit JUnit

Challenges

Setup und Teardown

Sie haben in den Challenges in “Intro SW-Test” erste JUnit-Tests für die Klasse `MyList<T>` implementiert.
Wie müssten Sie Ihre JUnit-Tests anpassen, wenn Sie im obigen Szenario Setup- und Teardown-Methoden einsetzen würden?

Parametrisierte Tests

Betrachten Sie die folgende einfache Klasse `MyMath`:

```
public class MyMath {
    public static String add(String s, int c) {
        return s.repeat(c);
    }
}
```

Beim Testen der Methode `MyMath#add` fällt auf, dass man hier immer wieder den selben Testfall mit lediglich anderen Werten ausführt - ein Fall für parametrisierte Tests.
Schreiben Sie mit Hilfe von JUnit (4.x oder 5.x) einige parametrisierte Unit-Tests für die Methode `MyMath#add`.

Quellen

Kleuker, S. 2019. *Qualitätssicherung durch Softwaretests*. Springer Vieweg. https://doi.org/10.1007/978-3-658-24886-4.

Osherove, R. 2014. *The Art of Unit Testing*. Manning.

Spillner, A., und T. Linz. 2012. *Basiswissen Softwaretest*. 5. Aufl. dpunkt.

The JUnit Team. 2022. „JUnit 5“. 2022. https://junit.org/.

Thies, A., C. Noelke, und Ungerc. o. J. „Einführung in JUnit“. Fernuniversität in Hagen. Zugegriffen 14. April 2020. https://wiki.fernuni-hagen.de/eclipse/index.php/EinfÃijhrung_in_JUnit.

vogella GmbH. 2021. „JUnit 5 Tutorial - Learn How to Write Unit Tests“. 2021. https://www.vogella.com/tutorials/JUnit/article.html.

Winters, T., T. Manshreck, und H. Wright. 2020. *Software Engineering at Google: Lessons Learned from Programming Over Time*. O’Reilly.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 2ce9f3f (Tooling: use original folder structure for generated pdf's, 2025-04-17)